COMPUTER ENGINEERING

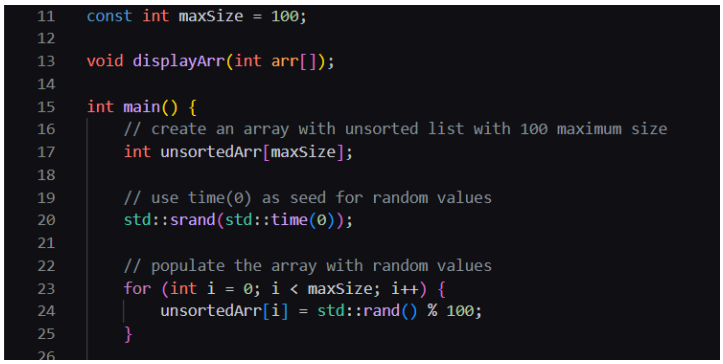
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	09/10/2026	**DATE SUBMITTED:**	09/10/2026
NAME:	Aguila, Daniel C.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA 7.1 Sorting Algorithms Pt1

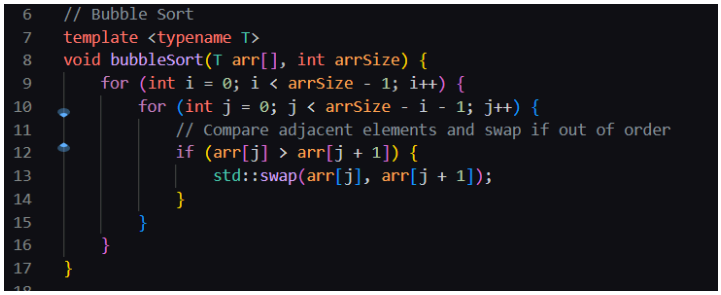
1. PROCEDURE OUTPUT

What procedures did you perform?

Code + Console Screenshot	
Observation/Analysis	unsortedArray is an array with 100 random values that have been generated using the expression rand() % 100 and thus the random values will not exceed 99. The use of srand(time(0)) ensures that there will be different random numbers in every run. Afterward, the for loop will be used to fill the array with the random values.

```
11  const int maxSize = 100;
12
13  void displayArr(int arr[]);
14
15  int main() {
16      // create an array with unsorted list with 100 maximum size
17      int unsortedArr[maxSize];
18
19      // use time(0) as seed for random values
20      std::srand(std::time(0));
21
22      // populate the array with random values
23      for (int i = 0; i < maxSize; i++) {
24          unsortedArr[i] = std::rand() % 100;
25      }
```

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observation/Analysis	The bubbleSort algorithm is made up of two for-loops where we compare the elements of the array in order to arrange them in increasing order. This works in a way that the outer loop picks out

```
6   // Bubble Sort
7   template <typename T>
8   void bubbleSort(T arr[], int arrSize) {
9       for (int i = 0; i < arrSize - 1; i++) {
10          for (int j = 0; j < arrSize - i - 1; j++) {
11              // Compare adjacent elements and swap if out of order
12              if (arr[j] > arr[j + 1]) {
13                  std::swap(arr[j], arr[j + 1]);
14              }
15          }
16      }
17  }
```

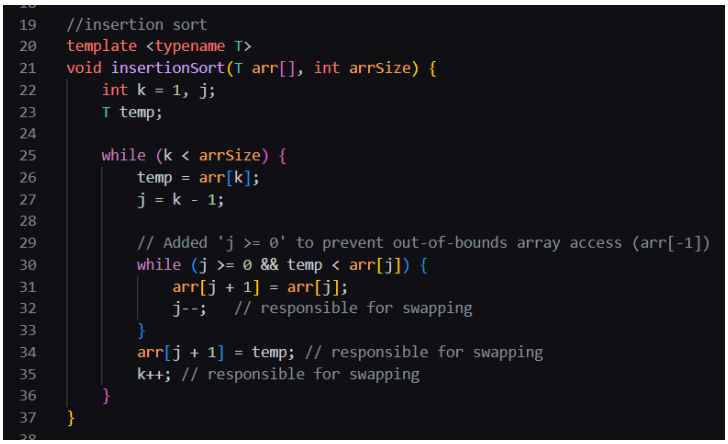
	an element to check, and then the inner loop checks the element picked against other elements. In case arr[i] is greater than arr[j], we exchange their values. Time complexity for the program is O(n^2) since there are nested loops that go to n and there is no exit or return in the loop.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	
Observation/Analysis	Selection sort algorithm consists of two loops; the outer loop which runs once for each element and calls the helper function to return the index of the minimum element in that iteration. Once found, it compares the element at the returned index with the current element in the outer loop and swaps them. The time complexity of this function is O(n^2) since the helper function is implemented by a for loop searching through the elements of the array at each iteration of the selectionSort loop.

```
39  //helper function
40
41  template <typename T>
42  int routine_smallest(T A[], int maxSize, int k){ //int k = current index;
43
44      int position, j;
45      T smallestElement = A[k];
46
47      for (int J = k+1; J < maxSize; J++) {
48          if (A[J] < smallestElement) {
49              smallestElement = A[J];
50              position = J;
51          }
52      }
53
54      return position;
55  }
56
57  //selection sort
58  template <typename T>
59  void selectionSort(T arr[], int arrSize) {
60      int POS, temp;
61
62      for (int i = 0; i < arrSize; i++) {
63          POS = routine_smallest(arr, arrSize, i);
64
65          temp = arr[i];
66          arr[i] = arr[POS];
67          arr[POS] = temp;
68
69      }
70
71  }
72
```

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	
Observation/Analysis	first element of the array in the InsertionSort function is always sorted, hence the algorithm begins its sorting process starting from the second element. In the sorting process, while loop is used instead of for loop in iterating through the elements of the array. A variable temp is declared to hold the value of the current element, while the inner loop compares it with the preceding element. Now, if temp is less than the preceding element, then the latter is shifted to the right until temp takes its proper place. The time complexity of the algorithm is O(n^2), since the outer while loop iterates through the elements, while the inner loop compares the elements and shifts them to the right.

```
19  //insertion sort
20  template <typename T>
21  void insertionSort(T arr[], int arrSize) {
22      int k = 1, j;
23      T temp;
24
25      while (k < arrSize) {
26          temp = arr[k];
27          j = k - 1;
28
29          // Added 'j >= 0' to prevent out-of-bounds array access (arr[-1])
30          while (j >= 0 && temp < arr[j]) {
31              arr[j + 1] = arr[j];
32              j--;   // responsible for swapping
33          }
34          arr[j + 1] = temp; // responsible for swapping
35          k++; // responsible for swapping
36      }
37  }
```

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Pseudocode

```
START
    DECLARE array size of 100
    DECLARE 5 candidates

    FOR i = 0 to 100
        GENERATE random vote from 1 to 5
        STORE vote in the array
    END FOR

    DISPLAY "Unsorted votes"
    DISPLAY the array

    DISPLAY "Sorted votes"
    SORT the array using the selection sort
    DISPLAY the array

    DECLARE currentVote as the first element of the array
    DECLARE count as 1

    FOR i = 1 to 100
        IF the current element is equal to currentVote
            INCREMENT count
        ELSE
            DISPLAY "Candidate" currentVote "received" count "votes."
            SET currentVote to the current element
            RESET count to 1
        END IF
    END FOR

    DISPLAY the total votes for each candidate
END
```

Main.cpp

```cpp
//Aguila, Daniel C. CPE21S1
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingSupp.h"

const int maxSize = 100;
const int numCandidates = 5;

int main(){
    int votes[maxSize];
    std::string candidates[numCandidates] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustín",
        "Deja Jayla Bañaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    srand(time(0));
    for(int i = 0; i < maxSize; i++){
        votes[i] = rand() % numCandidates + 1;
    }

    std::cout << "Unsorted votes: ";
    displayArray(votes, maxSize);
    std::cout << std::endl;

    std::cout << "Sorted votes: ";
    selectionSort(votes, maxSize);
    displayArray(votes, maxSize);
    std::cout << std::endl;

    countVotes(votes, maxSize);

    return 0;
}
```

Sorting.h

```cpp
//Aguila, Daniel C. CPE21S1
#ifndef SORTINGSUPP_H
#define SORTINGSUPP_H

//helper function
template <typename T>
int routine_smallest(T arr[], int k, int arrSize){
    int position = k;

    T smallestElement = arr[k];

    for(int j = k + 1; j < arrSize; j++){
        if(arr[j] < smallestElement){
            smallestElement = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], int arrSize){
    int min;

    for(int i = 0; i < arrSize; i++){
        min = routine_smallest(arr, i, arrSize);
        std::swap(arr[i], arr[min]);
    }
}

template <typename T>
void countVotes(T arr[], int arrSize){
    T currentVote = arr[0];
    int count = 1;
```

```cpp
template <typename T>
void countVotes(T arr[], int arrSize){
    T currentVote = arr[0];
    int count = 1;

    for(int i = 1; i < arrSize; i++){
        if(arr[i] == currentVote){
            count++;
        } else {
            std::cout << "Candidate " << currentVote << " received " << count << " votes." <<
            currentVote = arr[i];
            count = 1;
        }
    }
    std::cout << "Candidate " << currentVote << " received " << count << " votes." << std::en
}

template <typename T>
void displayArray(T arr[], int arrSize){
    for(int i = 0; i < arrSize; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

#endif
```

Output

```
[Running] cd "c:\Users\daniel\OneDrive\CPE21S1 Aguila\Supplementary\supp\" && g++ main.cpp -o main &&
"c:\Users\daniel\OneDrive\CPE21S1 Aguila\Supplementary\supp\"main
Unsorted votes: 5 4 3 2 5 3 5 5 1 1 2 2 3 5 1 2 1 5 1 3 2 5 1 1 1 4 3 2 1 2 4 2 3 2 1 4 3 4 3 1 1 4 3 2 4 2
3 3 2 3 4 2 3 1 5 4 2 4 3 3 1 2 4 1 5 3 2 1 4 4 2 4 4 2 3 2 2 4 3 5 2 5 2 2 4 4 5 2 3 1 4 1 1 3 4 1 4 2 4 2

Sorted votes: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3
3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5

Candidate 1 received 20 votes.
Candidate 2 received 26 votes.
Candidate 3 received 20 votes.
Candidate 4 received 22 votes.
Candidate 5 received 12 votes.
```

Question: Was your developed vote counting algorithm effective? Why or why not?
Yes, the vote counting algorithm I implemented worked efficiently as doing sorting at first ensured that the votes for each candidate would be together, thus allowing the countVotes() function to do the counting operation in a sequential order and not by iterating the array again and again. I selected the Selection Sort because it uses the least_sma routine function that finds the least value first before swapping it to the proper position. This way is

optimal as far as I know because we have to work only with the values from 1 to 5. The complexity of the Selection Sort is O(n²) because it uses nested loops for comparisons, while for the voting counting algorithm it is O(n).

Conclusion :
I have learned how to create a sorting algorithm and how this particular sorting algorithm can be used to sort a large number of array values. From this activity, I have learned about how to use the proper bubble sort, insertion sort and selection sort. The bubble sort compares the two adjacent elements and swaps them, the insertion sort takes an element and puts it at its rightful place considering that the first index is sorted and selection sort takes the smallest value and sorts it into the right place. As for the extra credit activity, it was challenging but not in any difficult way because we had a sorting algorithm that needs to be implemented. Finally I have been able to solve this activity by using a for loop which will iterate over the array and find out the number of appearances for each candidate. Overall I think I have done a great job in this activity because I have managed to create and implement the sorting algorithm and not hard coding.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.